



University of Sialkot

Semester Project – Deliverable 2 (Server Side Architecture)

Group Members

Kamil Ahmed: 23010101-024

Husnain Khizer: 23010101-029

TOTAL GRADE POINT 10

GRADE POINT ALLOCATED

ASSESSOR'S COMMENTS

Instructor: Museb Khalid

Abstract

Blood Donor Finder is a full-stack web application that connects blood donors with people who urgently need blood. Donors register their personal and medical details once, and seekers can instantly search for available donors by blood group and city. The system is built using Vue 3 and Vite for the frontend, Node.js and Express.js for the backend REST API, and MongoDB Atlas as the cloud database. The frontend implements Single Page Application routing, modular component architecture, and bidirectional inter-component communication through props and custom events. The backend provides five RESTFUL API endpoints handling donor registration, retrieval, search, phone-based profile lookup, and availability status updates. This report documents the complete architecture, features, and design decisions of both the client-side and server-side components of the application.

I. Introduction

Finding compatible blood in an emergency is a serious and time-sensitive problem. Patients and their families often rely on slow word-of-mouth or unmoderated social media posts to locate a willing donor. Blood Donor Finder solves this problem by providing a centralized, searchable system where donors register once and seekers can find them instantly.

It demonstrates the practical implementation of a full-stack web application using modern technologies including Vue 3, Express.js, and MongoDB. The system is divided into two parts: a client-side frontend that handles all user interaction, and a server-side backend that manages data storage and retrieval through a REST API.

II. System Overview

Blood Donor Finder follows a three-tier architecture consisting of a presentation layer (Vue 3 frontend), a logic layer (Express.js backend API), and a data layer (MongoDB Atlas database). The frontend and backend run as two separate servers during development and communicate exclusively through HTTP requests using the Axios library.

Layer	Technology	Port
Frontend	Vue 3 + Vite	localhost:5173
Backend	Node.js + Express.js	localhost:5000
Database	MongoDB Atlas	Cloud

Table 1. Three-tier architecture of Blood Donor Finder.

III. Technology Stack

Technology	Purpose
Vue 3	Frontend JavaScript framework
Vite	Development server and build tool
Vue Router 4	Single Page Application routing
Axios	HTTP requests from frontend to backend
Plain CSS	Styling and responsive layout
Node.js	Server-side JavaScript runtime
Express.js	REST API framework
MongoDB Atlas	Cloud NoSQL database
Mongoose	MongoDB schema and query library
dotenv	Environment variable management
CORS	Cross-origin request handling
Nodemon	Auto-restart during development

Table 2. Complete technology stack.

IV. Features

A. Donor Registration

Any person who wants to donate blood can register through the registration form. The form collects the donor's full name, age, gender, blood group, city, phone number, and last donation date. The autocomplete attribute is disabled on all form fields to protect the privacy of previous users. When the form is submitted, the data is sent to the backend and saved permanently in MongoDB. The donor is automatically marked as available upon registration.

B. Search Donors

A seeker who needs blood can visit the Find a Donor page and search by selecting a blood group and typing a city name. The system queries the backend which filters donors by blood group and city using a case-insensitive search. Only donors who are currently marked as available are displayed in the results. Each result card shows the donor's name, blood group, city, age, gender, and phone number.

C. Update Availability Status

After donating blood, a donor can visit the Update My Status page and enter their registered phone number. The system finds their profile and displays their current availability status. The donor can then mark themselves as unavailable. When they are ready to donate again after recovery, they can return and mark themselves as available. This feature ensures that seekers only see donors who are genuinely able to donate.

D. All Donors Page

An All Donors page displays all currently available donors in the system. This page automatically fetches fresh data from the database every time it is visited, so any availability changes made through the Update My Status page are reflected immediately without requiring a manual page refresh.

V. Use Cases

Use Case	Actor	Description
Register as Donor	Donor	Donor fills registration form with personal and blood group details. Data is saved to MongoDB.
Search for Blood	Seeker	Seeker selects blood group and enters city. System returns list of available matching donors.
View All Donors	Seeker	Seeker browses all currently available donors in the system.
Mark Unavailable	Donor	Donor enters phone number, finds profile, and marks themselves as unavailable after donating.
Mark Available Again	Donor	After recovery, donor returns and marks themselves as available again.

Table 3. Use cases of Blood Donor Finder.

VI. Frontend Architecture

A. Project Structure

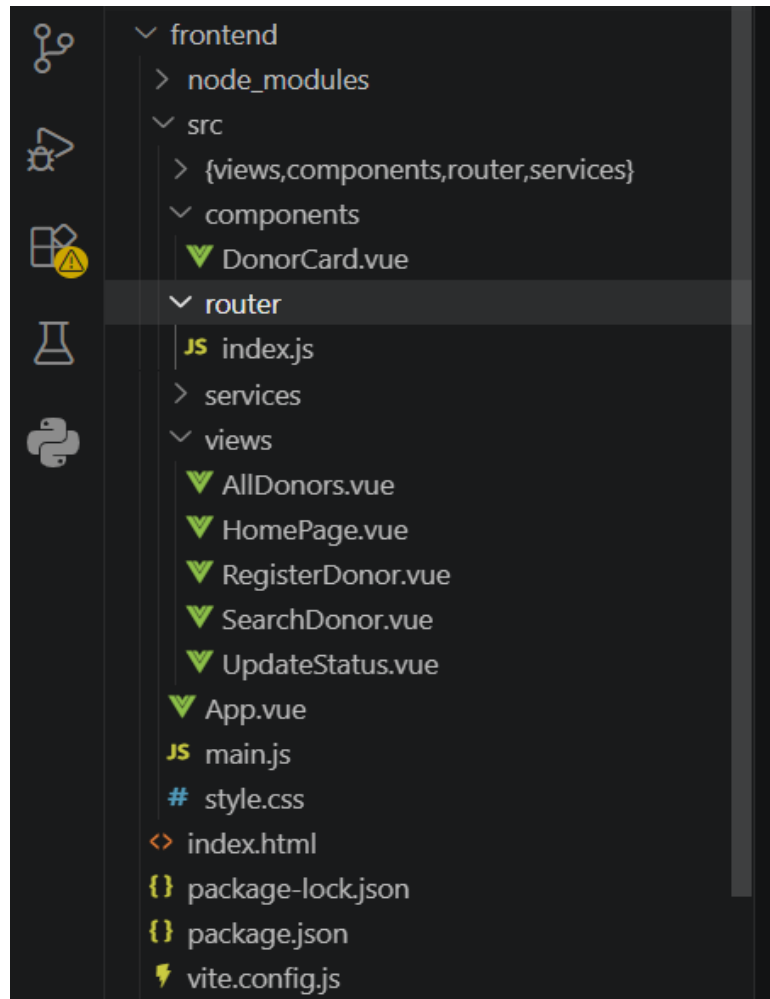


Fig. 1. Frontend folder structure.

B. Single Page Application Routing

Vue Router is used to handle navigation between the five pages of the application without reloading the browser. Each route maps a URL path to a Vue component. The App.vue component contains the persistent navigation bar and a RouterView outlet where the active page is rendered. RouterLink components replace standard anchor tags to prevent full page reloads.

C. Modular Component Architecture

The user interface is divided into two types of Vue components. Views are full pages that map to routes: HomePage, RegisterDonor, SearchDonor, AllDonors, and UpdateStatus. Components are

smaller reusable pieces DonorCard is the only reusable component and it is used in both the SearchDonor and AllDonors pages to display a single donor's information consistently.

D. Vue Concepts Used

Concept	Where Used
ref()	Loading flags, error messages, search inputs
reactive()	Registration form object with multiple fields
v-model	Two-way binding on all form inputs
v-for	Rendering donor list and blood group options
v-if / v-else	Showing loading, results, or empty state
onMounted()	Auto-fetching donors when AllDonors page loads
Props	Passing donor data from parent to DonorCard
async/await	All backend API calls with try/catch error handling

Table 4. Vue concepts used in the project.

E. API Service Layer

All HTTP communication with the backend is centralized in a single file called donorService.js. This means Vue components never call Axios directly — they only call service functions. This separation of concerns makes the code easier to maintain and debug because any API changes only need to be made in one place.

VII. Backend Architecture

A. Project Structure

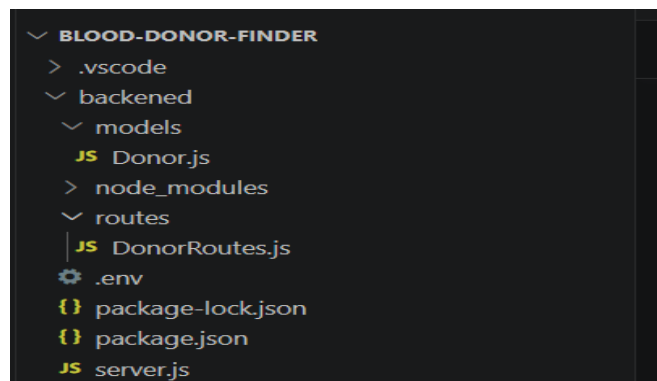


Fig. 2. Backend folder structure.

B. Database Schema

The Donor schema defines the structure of every donor document stored in MongoDB. A schema is a blueprint that enforces what fields each record must have and what data type each field must contain. The timestamps option tells Mongoose to automatically add createdAt and updatedAt fields to every document.

```
const mongoose = require('mongoose')

const donorSchema = new mongoose.Schema({
  name:      { type: String, required: true },
  age:       { type: Number, required: true },
  gender:    { type: String, required: true },
  bloodGroup: { type: String, required: true },
  city:      { type: String, required: true },
  phone:     { type: String, required: true },
  lastDonationDate: { type: Date },
  available:  { type: Boolean, default: true }
}, { timestamps: true })

module.exports = mongoose.model('Donor', donorSchema)
```

Fig. 3, Mongoose schema for the Donor model.

Field	Type	Required	Description
name	String	Yes	Full name of the donor
age	Number	Yes	Age of the donor
gender	String	Yes	Male / Female / Other
bloodGroup	String	Yes	A+, A-, B+, B-, O+, O-, AB+, AB-
city	String	Yes	City or area of the donor
phone	String	Yes	Registered phone number
lastDonationDate	Date	No	Date of last blood donation
available	Boolean	No	Availability status, default: true

Table 5. Donor schema fields and descriptions.

C. REST API Endpoints

Method	Endpoint	Description
POST	/api/donors	Register a new donor
GET	/api/donors	Get all donors
GET	/api/donors/phone/:phone	Find donor by phone number
GET	/api/donors/search	Search by blood group and city
PUT	/api/donors/:id	Update donor availability status

Table 6. REST API endpoints.

VIII. Data Flow

A. Registration Flow

The donor fills the registration form in the Vue frontend. When the form is submitted, Axios sends a POST request to /api/donors with the form data. Express receives the data in req.body, creates a new Donor object using the Mongoose model, and saves it to MongoDB. A 201 success response is sent back to the frontend which then displays a success message and clears the form.

B. Search Flow

The seeker selects a blood group and types a city on the Search page. Axios sends a GET request to /api/donors/search with these values as query parameters. The backend builds a filter object and uses MongoDB's \$regex operator for case-insensitive city matching. The results array is returned to the frontend which filters it to show only donors where available is true, then renders a DonorCard for each result.

C. Status Update Flow

The donor visits the Update My Status page and enters their registered phone number. Axios sends a GET request to /api/donors/phone/:phone and the backend uses findOne() to locate the matching record. The donor's profile is displayed with their current status. When the donor clicks Mark Unavailable, Axios sends a PUT request to /api/donors/:id with the new availability value. The backend uses findByIdAndUpdate() to update the record in MongoDB.

IX. Security Considerations

- Environment variables — MongoDB credentials stored in .env, never in source code.
- CORS configuration — only allows requests from the frontend origin.
- Input validation — Mongoose schema enforces required fields and data types, rejecting malformed data with a 400 response.
- Phone-based identity — donors update their own status using their registered phone number, preventing unauthorized status changes by other users.

X. Conclusion

Blood Donor Finder is a complete full-stack web application that successfully addresses the real-world problem of finding blood donors quickly in an emergency. The system provides three core features: donor registration, blood search by group and city, and self-managed availability status updates.

The frontend demonstrates core Vue 3 concepts including Single Page Application routing, modular component architecture, props and emits for inter-component communication, reactive state management, and async API communication. The backend provides a clean REST API built with Express.js, a well-structured Mongoose schema, and secure credential management using environment variables. MongoDB Atlas provides reliable cloud-hosted data persistence accessible from any network. The project demonstrates a practical understanding of full-stack web development and the separation of concerns between the client, server, and database layers.

References

- [1] Vue.js Core Team, "Vue.js Documentation," vuejs.org, 2024. [Online]. Available: <https://vuejs.org/guide/>
- [2] Vue Router Team, "Vue Router Documentation," router.vuejs.org, 2024. [Online]. Available: <https://router.vuejs.org/>
- [3] Evan You et al., "Vite Guide," vitejs.dev, 2024. [Online]. Available: <https://vitejs.dev/guide/>
- [4] OpenJS Foundation, "Express.js Guide," expressjs.com, 2024. [Online]. Available: <https://expressjs.com/>
- [5] MongoDB Inc., "Mongoose Documentation," mongoosejs.com, 2024. [Online]. Available: <https://mongoosejs.com/docs/>
- [6] MongoDB Inc., "MongoDB Atlas," mongodb.com, 2024. [Online]. Available: <https://www.mongodb.com/docs/atlas/>